

Modul Media Programming, Lehrveranstaltung AV Programming



Prof. Dr. Eva Wilk

HAW Hamburg, Fakultät INF, Studiengang Medieninformatik

Termine

1: Audio - Signalweg und digitale Repräsentation in Python	21.04.2026
2: Video - Signalweg und digitale Repräsentation in Python	05.05.2026
3: Audioformate und Audio-Datenreduktion	19.05.2026
4: <i>Test 1 zu Media-Programming</i> Videoformate und Video-Datenreduktion	02.06.2026
5: AV-Pipeline und Synchronisation	16.06.2026
6: <i>Test 2 zu Media Programming</i> AV-Bearbeitung und Synchronisation	30.06.2026
7: Audio- und Video-Inhaltsanalyse Wiederholung	14.07.2026

Thema 2 - Übersicht

- Vom Bildsignal zum Frame
- Video-Eigenschaften: Auflösung, Framerate, Framezahl, Bildgröße
- Video in Python öffnen: einzelne Frames lesen
- Farbe, Kanäle, Graustufen
- Zusammenfassung
- Übungsaufgaben zum 18.5.2026

Thema 2 - Übersicht

Ein Video besteht aus einer Folge digitaler Einzelbilder.

Diese Bilder entstehen nicht direkt „in Python“, sondern zunächst durch optische Erfassung mit einer Kamera bzw. durch Decodierung einer Videodatei.

In Python/OpenCV erscheint ein Frame dann als Array mit Höhe, Breite und — bei Farbbildern — einer Kanalstruktur.

Damit wird ein technischer Zusammenhang sichtbar: Zwischen realer Szene und Programm stehen immer Sensorik, Digitalisierung und Repräsentation.

Thema 2 - Lernziel

Die Studierenden erläutern den Übergang von visuellen Szenen zu digital repräsentierten Bild- und Videodaten und lesen Videodaten in Python/OpenCV ein, extrahieren Frames und wenden einfache Operationen an.

Vom Bild zum Film zum 4K-Video (sehr kurz)

Wahrgenommene Bewegung entsteht aus einer zeitlich geordneten Folge einzelner Bilder.

Weiterentwicklung: Aufnahmetechnik, Material, Bildgröße, Speicherung und Auflösung.

- frühe Filmprojektion: oft ca. 16 fps
- Tonfilmstandard: 24 fps
- 35 mm = klassisches Kinoformat
- 16 mm = Unterricht, Doku, semiprofessionell
- Super 8 = Amateur- und Home-Movie-Format
- Super 16 = größere nutzbare Bildfläche als 16 mm
- heute: überwiegend digitales Video

Vom Bild zum Film zum 4K-Video (sehr kurz)

- Bewegungswahrnehmung ab ca. 16 fps
- 24 fps = filmischer Standard
- 50/60 fps = meist deutlich glatter
- es gibt keine feste „magische“ fps-Zahl
- Wahrnehmung hängt auch von Helligkeit und Blicksituation ab
- 4K = ca. 4.000 Pixel Bildbreite
- Heimvideo meist 3840 × 2160 (UHD)
- Kino oft 4096 × 2160 (DCI 4K)

Abschnitt 1: Von der Szene zum digitalen Frame

- reale Szene: optisches Signal
 - Kamera: Gesamtsystem der Bilderfassung
 - Bildsensor: lichtempfindliche Pixelmatrix
 - Auslesen: Überführen Sensorsignale in elektrische Signalfolge
 - Digitalisierung: Umwandlung dieser analogen Signale in diskrete Zahlenwerte
 - Farbe: Farbfilter und anschließende Rekonstruktion
 - digitales Einzelbild (Frame)
 - Frame in Python

Vom Bildsignal zum digitalen Frame

Ein Video beginnt nicht als Datei, sondern als visuelle Szene.

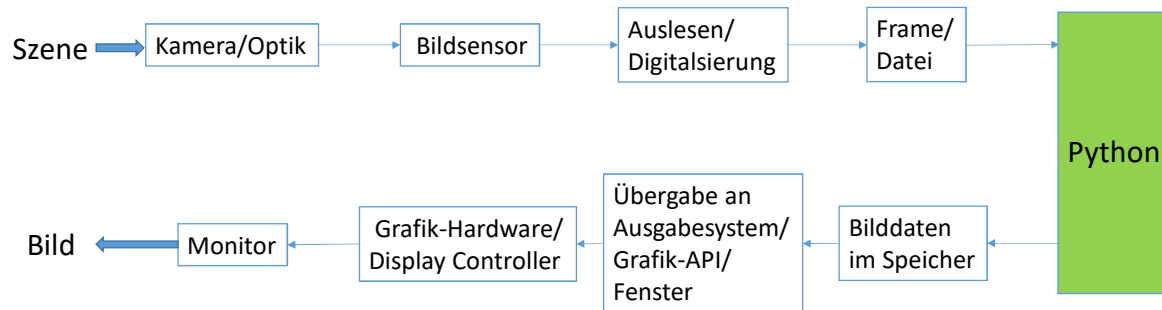
Die Kamera erfasst Licht über den Sensor; dabei beeinflussen Sensorauslesung und Kameraeinstellungen, wie schnell Bilder entstehen können.

In Python/OpenCV erscheint das Ergebnis später als Folge digitaler Frames.

Vom Frame zum Bildsignal

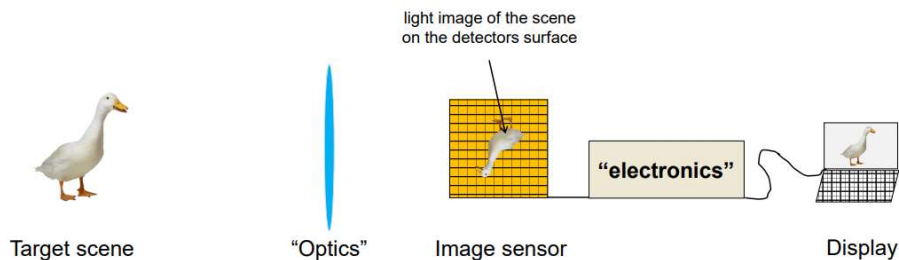
- Daten in Python
 - Bilddaten im Speicher
 - Übergabe an Ausgabesystem / Grafik-API / Fenster
 - Grafik-Hardware / Display-Controller
 - Monitor
 - sichtbares Bild

Vom Bildsignal zum Frame zum Bildsignal



The idea of imaging

HAMAMATSU
PHOTON IS OUR BUSINESS



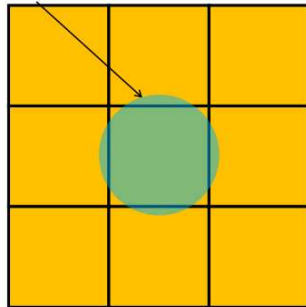
1. The role of an image sensor is to convert the imaged light scene to an electronic image
2. The conversion of light's energy to electrical signal occurs in a pixel
3. Pixel is the smallest imaging element in an image sensor

https://www.hamamatsu.com/content/dam/hamamatsu-photonics/sites/static/hc/resources/webinars/Introduction_to_image_sensors.pdf

From photons to electrons

HAMAMATSU
PHOTON IS OUR BUSINESS

imaged element of
the scene



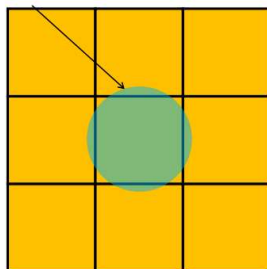
3 × 3 array of pixels

no electrical signal	little electrical signal	no electrical signal
little electrical signal	a lot of electrical signal	little electrical signal
no electrical signal	little electrical signal	no electrical signal

From photons to electrons

https://www.hamamatsu.com/content/dam/hamamatsu-photonics/sites/static/hc/resources/webinars/introduction_to_image_sensors.pdf

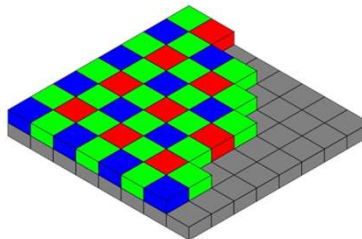
imaged element of
the scene



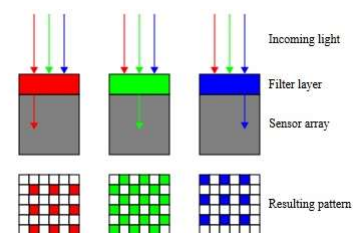
3 × 3 array of pixels

no electrical signal	little electrical signal	no electrical signal
little electrical signal	a lot of electrical signal	little electrical signal
no electrical signal	little electrical signal	no electrical signal

Of each four pixels, two pixels are given a green colour filter, one pixel a red filter and one pixel a blue colour filter. This colour distribution corresponds to the perception of the human eye and is referred to as the Bayer matrix. A pixel depicts only the information for one colour.



The principle of digital image sensors means that they acquire only brightness – but not colour – information. As a result, a colour filter is applied to each pixel during manufacture of colour sensors. This is known as the Bayer matrix.



<https://en.ids-imaging.com/techtipp-details/items/techtipp-18mp-color-sensor-as-mono.html>

Was ist ein Frame in Python?

- Frame: Bild als Raster aus Pixeln
- Höhe, Breite, ggf. Kanalzahl
- Farbbild vs. Graubild

In OpenCV lassen sich Bildeigenschaften direkt über `img.shape` ansehen. Bei Farbbildern liefert `shape` Zeilen, Spalten und Kanäle; bei Graubildern nur Zeilen und Spalten. Daran wird die digitale Repräsentation unmittelbar sichtbar.

Einschub: Was ist OpenCV?

- OpenCV = Open Source Computer Vision Library
- gestartet 1999 bei Intel, erste Veröffentlichung 2000
- OpenCV-Python = Python-API bzw. Python-Bindings für die zugrunde liegende C++-Bibliothek
- Bild- und Videodaten werden in Python als NumPy-Arrays verarbeitet.
- unterstützt viele Verfahren aus Computer Vision und Machine Learning
- plattformübergreifend, u. a. für Windows, Linux, macOS, Android, iOS

https://docs.opencv.org/4.x/d0/de3/tutorial_py_intro.html

Einschub: Was ist OpenCV?

- Bilddaten erscheinen in Python als NumPy-Arrays
- wichtige Grundbegriffe:
 - `img.shape` zeigt Höhe, Breite, Kanalzahl
 - OpenCV arbeitet standardmäßig mit BGR, nicht RGB
 - Videos/Kameras werden über VideoCapture geöffnet und frameweise gelesen
- Für diesen Termin besonders relevant:
 - Bilder/Videos laden
 - Frames lesen
 - Farben umwandeln
 - einfache Operationen nachvollziehen

Beispiel 5a: Video öffnen und ersten Frame lesen

- `cv.VideoCapture(...)` # Class for video capturing from video files, image sequences or cameras.
 # The class provides C++ API for capturing video from cameras or for reading video files
 # and image sequences.
- `cap.isOpened()` # You can check whether the capture is initialized or not by the method
 # `cap.isOpened()`. If it is True, OK. Otherwise open it using `cap.open()`.
- `ret, frame = cap.read()` # returns a bool (True/False).
 # If the frame is read correctly, it will be True. So you can check for the end of the video
 # by checking this returned value.
- `frame.shape` # The shape of an image is accessed by `img.shape`. It returns a tuple of the number
 # of rows, columns, and channels (if the image is color):

Beispiel 5a: Video öffnen und ersten Frame lesen

```
import cv2 as cv

cap = cv.VideoCapture("video.mp4")

if not cap.isOpened():
    print("Video konnte nicht geöffnet werden.")
    raise SystemExit

ret, frame = cap.read()

if ret:
    print("Shape des ersten Frames:", frame.shape)

cap.release()
```

Testvideos finden Sie hier: <https://test-videos.co.uk> , <https://www.pexels.com/video/ocean-waves-19573481>
<https://www.pexels.com/video/traffic-jam-on-street-14552311>

Beispiel 5b: Kamera öffnen und ersten Frame lesen

```
import cv2 as cv

cap = cv.VideoCapture(0) # interne Kamera: 0, erste USB-Kamera: 1, weitere: 2,3,...
if not cap.isOpened():
    print("Kamera konnte nicht geöffnet werden.")
    raise SystemExit

ret, frame = cap.read() # ret: False oder True
if not ret:
    raise SystemExit
print("Erster Frame:", frame.shape)
while True:
    ret, frame = cap.read()
    if not ret:
        print("Frame konnte nicht gelesen werden.")
        break
    cv.imshow("Kamerabild", frame)
    if cv.waitKey(1) == ord("q"): # Achtung: q im Kamerabild drücken
        break
cap.release() # Verbindung zur Kamera wird beendet, andere Prg. können darauf zugreifen
cv.destroyAllWindows()
```

Bild-Eigenschaften

- Rasterbild: Ein Bild, das aus einem regelmäßigen Gitter einzelner Pixel besteht. Jedem Pixel sind Werte zugeordnet, z. B. Helligkeit oder Farbkanäle.
- Auflösung: Die Anzahl der Pixel in Breite und Höhe eines Bildes, z. B. 1280×720 . Sie beschreibt, wie fein ein Rasterbild räumlich aufgeteilt ist.
- Mehrkanalbild: Ein Bild, bei dem pro Pixel mehrere Werte gespeichert werden, z. B. drei Kanäle für Farbe. In OpenCV ist das typischerweise ein BGR-Bild mit drei Kanälen.

Abschnitt 2: Video-Eigenschaften und Zeitbezug

- Auflösung (Breite/Höhe)
- Framerate (fps)
- Framezahl
- Dauer eines Videos
- Unterschied zwischen Bildfolge und Zeitachse

Ein Video ist nicht nur eine Sammlung von Bildern, sondern auch eine zeitlich geordnete Folge. Für die Verarbeitung sind deshalb nicht nur Auflösung und Kanalzahl wichtig, sondern auch Framerate und Framezahl. Erst aus diesen Angaben lässt sich der Zeitbezug eines Videos erschließen.

OpenCV stellt über Video-I/O-Eigenschaften u. a. Framebreite, Framehöhe, Framerate und Anzahl der Frames bereit. Die API unterscheidet zwischen der aktuellen Position in Millisekunden und der aktuellen Position in Frames.

Beispiel 6: Eigenschaften des Videos – Metadaten lesen

- `cv.CAP_PROP_FRAME_WIDTH` # Width of the frames in the video stream.
- `cv.CAP_PROP_FRAME_HEIGHT` # Height of the frames in the video stream.
- `cv.CAP_PROP_FPS` # Frame rate.
- `cv.CAP_PROP_FRAME_COUNT` # Number of frames in the video file.

OpenCV-Befehle sh.:

https://docs.opencv.org/4.x/d4/d15/group_videoio_flags_base.html#ggaeb8dd9c89c10a5c63c139bf7c4f5704dab26d2ba37086662261148e9fe93eecd

Beispiel 6: Eigenschaften des Videos

```
import cv2 as cv

cap = cv.VideoCapture("video.mp4")

if not cap.isOpened():
    print("Video konnte nicht geöffnet werden.")
    raise SystemExit

width = cap.get(cv.CAP_PROP_FRAME_WIDTH) # Breite der Frames im Video-Stream
height = cap.get(cv.CAP_PROP_FRAME_HEIGHT) # Höhe der Frames im Video-Stream
fps = cap.get(cv.CAP_PROP_FPS) # Frame rate
frame_count = cap.get(cv.CAP_PROP_FRAME_COUNT) # Anzahl der Frames im Video-File

print(f"Breite: {width:.0f}") # f-String: 0: keine Nachkommastelle, f: float
print(f"Höhe: {height:.0f}")
print(f"FPS: {fps:.2f}")
print(f"Frames: {frame_count:.0f}")

cap.release()
```

Aufgabe 5

Ergänzen Sie den Code so, dass die geschätzte Dauer des Videos berechnet und ausgegeben wird.

Dauer: 3 Minuten

Abschnitt 3: Frameweises Lesen und Extraktion

Ein Video wird frame-weise gelesen:

- Videoverarbeitung als Schleife über Frames: `while`-Schleife
- sequentielles Lesen: `read()` bis `ret == False` (dann ist Ende des Videos erreicht)
- „jedes n-te Frame“ speichern: einfach, aber nicht automatisch zeitbasiert

https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html

Beispiel 7: Jedes 30. Frame speichern

```
import cv2 as cv

cap = cv.VideoCapture("video.mp4")
if not cap.isOpened():
    print("Video konnte nicht geöffnet werden.")
    raise SystemExit

frame_nr = 0
saved_nr = 0
while True:
    ret, frame = cap.read()
    if not ret:
        break
    if frame_nr % 30 == 0:
        filename = f"frame_{saved_nr:03d}.jpg"
        cv.imwrite(filename, frame)
        print("gespeichert:", filename)
        saved_nr += 1
    frame_nr += 1
cap.release()
```

cap.read() returns a bool (True/False). If the frame is read correctly, it will be True. So you can check for the end of the video by checking this returned value.

Speichern einzelner Frames

https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html

Beispiel 7: Jedes 30. Frame speichern

Wenn ein Video 30 fps hat, bedeutet „jedes 30. Frame“ ungefähr den Abstand einer Sekunde.

Bei 25 fps oder 60 fps gilt das nicht.

OpenCV unterscheidet dafür explizit zwischen `CAP_PROP_POS_FRAMES` und `CAP_PROP_POS_MSEC`

https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html

Abschnitt 4: Farbe, Kanäle, Graustufen

- Farbbilder als Mehrkanalbilder
- OpenCV: BGR, nicht RGB
- Graustufen als reduzierte Repräsentation und für Analyse
- `cvtColor()` für Farbkonvertierungen

Ein Farbbild enthält mehr Informationen als ein Graubild, weil mehrere Kanäle gespeichert werden.

In OpenCV ist die Standardreihenfolge der Farbkanäle BGR (!!)

Für viele einfache Analyseaufgaben reicht eine Graustufendarstellung aus, weil dabei nur Helligkeitswerte betrachtet werden. Dadurch wird die Datenrepräsentation einfacher, ohne dass die geometrische Struktur des Bildes verloren geht.

Beispiel 8: Farbframe in Graustufen umwandeln

```
import cv2 as cv

cap = cv.VideoCapture("video.mp4")

if not cap.isOpened():
    print("Video konnte nicht geöffnet werden.")
    raise SystemExit

ret, frame = cap.read()
if not ret:
    print("Kein Frame lesbar.")
    cap.release()
    raise SystemExit

gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY) # convert between RGB/BGR and grayscale,
print("Farbbild:", frame.shape)
print("Graubild:", gray.shape)
cv.imwrite("frame_color.jpg", frame)
cv.imwrite("frame_gray.jpg", gray)
cap.release()
```

Beispiel 8: Farbframe in Graustufen umwandeln

GRB -> GRAY

Transformations within GRB space like adding/removing the alpha channel, reversing the channel order, conversion to/from 16-bit RGB color (G6:R5:B5 or G5:R5:B5), as well as conversion to/from grayscale using:

$$\text{GRB[A] to Gray: } Y \leftarrow 0.587 \cdot G + 0.299 \cdot R + 0.114 \cdot B$$

The conversion from a RGB image to gray is done with:

```
cvtColor(src, bwsrc, cv::COLOR_RGB2GRAY);
```

https://docs.opencv.org/4.x/de/d25/imgproc_color_conversions.html#color_convert_rgb_gray

Aufgabe 6

Ergänzen Sie den Code so, dass zusätzlich Breite, Höhe und Kanalstruktur von Farb- und Graubild getrennt ausgegeben werden.

Was bleibt gleich, was ändert sich?

Dauer: 4 Minuten

Beispiel 9: Demonstration BGR (und SW)

```
import cv2 as cv
import numpy as np

# Leere Bilder erzeugen
breite = 200
hoehe = 200
blue = np.zeros((hoehe, breite, 3), dtype=np.uint8)
green = np.zeros((hoehe, breite, 3), dtype=np.uint8)
red = np.zeros((hoehe, breite, 3), dtype=np.uint8)

# Jeweils nur einen Kanal auf 255 setzen
blue[:, :, 0] = 255 # B-Kanal alle Pixel werden auf 255 gesetzt im 0. Kanal.
green[:, :, 1] = 255 # G-Kanal
red[:, :, 2] = 255 # R-Kanal

print("Blau-Pixel [B,G,R]:", blue[0, 0]) # erste beiden Dimensionen fest ausgewählt.
# Übrig bleibt dann nur noch die 3. Dimension = die 3 Kanalwerte dieses Pixels.
...
cv.imshow("B-Kanal -> Blau", blue)
```

Abschnitt 5: Einordnung

Zusammenfassung:

- reale Szene
- Erfassung durch Kamera/Sensor (← Auslesezeit, Bildrate)
- Datei bzw. Videostrom
- Decoder / OpenCV
- Frame als digitales Objekt in Python

Grenzen der drei Beispiele:

- noch keine echte Analyse
- noch keine Synchronisation
- noch keine komplexe Pipeline

Von der Szene bis zum Frame in Python

Codebeispiele: zeigen nur einen kleinen Ausschnitt des gesamten Videosystems.

Kameraeinstellungen und Sensoreigenschaften beeinflussen die resultierende Bildrate (Sensorauslesezeit, resultierende *Acquisition Frame Rate*)

OpenCV: arbeitet mit digital vorliegenden Frames, Datei-, Stream- oder Kamerazugriff

sh. auch: <https://docs.baslerweb.com/sensor-readout-time>

Sicherung und Ausblick

Wichtige Begriffe:

- Video
- Frame
- Auflösung
- Framerate, Framezahl
- Kanal
- BGR
- Graustufe

Quellen zu Thema 2

- Basler Product Documentation – Acquisition Frame Rate / Resulting Acquisition Frame Rate / Sensor Readout Time. Gute ergänzende Quellen für den kameraseitigen Realweltbezug: Bildrate als Kameraeigenschaft, resultierende Bildrate unter realen Bedingungen und Sensorauslesezeit. <https://docs.baslerweb.com/acquisition-frame-rate>
- Bayer: More resolution for color sensor. <https://en.ids-imaging.com/techtipp-details/items/techtipp-18mp-color-sensor-as-mono.html>
- Hamamatsu: Introduction to Images Sensors. https://www.hamamatsu.com/content/dam/hamamatsu-photonics/sites/static/hc/resources/webinars/Introduction_to_image_sensors.pdf
- OpenCV – Getting Started with Videos. Grundablauf für VideoCapture, read(), isOpened(), release(), Kamera- und Dateiquellen, Graustufenumwandlung im Videokontext. https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html
- OpenCV – VideoCapture Class Reference. Referenz zu VideoCapture als Klasse für Video-Dateien, Bildsequenzen oder Kameras. https://docs.opencv.org/4.x/d8/dfe/classcv_1_1VideoCapture.html
- OpenCV – Flags for video I/O. Referenz zu CAP_PROP_FRAME_WIDTH, CAP_PROP_FRAME_HEIGHT, CAP_PROP_FPS, CAP_PROP_FRAME_COUNT, CAP_PROP_POS_MSEC, CAP_PROP_POS_FRAMES. https://docs.opencv.org/4.x/d4/d15/group__videoio__flags__base.html
- OpenCV – Basic Operations on Images. Erklärung zu img.shape, Zeilen, Spalten, Kanälen und dem Unterschied zwischen Farb- und Graubild. https://docs.opencv.org/4.x/d3/df2/tutorial_py_basic_ops.html
- OpenCV – Color Space Conversions. Referenz zu cvtColor() und Hinweis, dass OpenCV standardmäßig BGR statt RGB verwendet. https://docs.opencv.org/4.x/d8/d01/group_imgproc_color_conversions.html
- Video Analysis with OpenCV and Python: A Comprehensive Guide <https://codezup.com/video-analysis-opencv-python/>